

Qwen3.8-27B

导论

Qwen3.8-27B 是 Qwen3.8 系列中的 27B 稠密模型。理解这一模型时，首先应关注它相对于传统 Decoder-only Transformer 的结构变化。传统大语言模型通常在每一层都使用完整的 Self-Attention；Qwen3.8-27B 的语言主干采用混合式 Decoder：64 层中有 48 层使用 Gated DeltaNet，另外 16 层使用 Gated Full Attention，并按照“三层 DeltaNet 加一层 Full Attention”的方式周期性排列。

这种设计把多种信息传播方式组合在同一条计算路径中。DeltaNet 使用固定尺寸的递归状态压缩历史信息，并在状态更新之前通过短程因果卷积提取局部模式；Full Attention 每隔几层重新建立一次显式的全局 Token 联系。因此，从功能上看，Qwen3.8-27B 同时包含局部卷积、递归记忆和全局注意力三种信息传播方式。

Qwen3.8-27B 还是一个原生视觉语言模型。文本经过 Tokenizer 和 Token Embedding 得到语言 Token，图片或视频经过 Vision Encoder 与 Patch Merger 得到视觉 Token，二者最终都被映射到 5120 维隐藏空间，再一起送入同一套 64 层语言主干。因此后续讲解可以按照真实的数据流顺序进行：先讨论文本输入，再讨论视觉输入，然后介绍语言主干中的两类 Token Mixer，最后说明 FFN、输出层以及推理缓存。

一、模型整体结构

Qwen3.8-27B 的语言隐藏维度为 $d = 5120$ ，语言层数为 $L = 64$ ，词表大小为 $V = 248320$ ，FFN 中间维度为 $d_{\text{ff}} = 17408$ 。语言主干以四层作为一个基本周期：前三层使用 Gated DeltaNet，第四层使用 Gated Full Attention，这个周期重复 16 次。因此可以直接得到 48 个 DeltaNet 层和 16 个 Full Attention 层。

每一层虽然使用的 Token Mixer 不同，但外部结构是统一的。输入首先经过 RMSNorm，然后进入当前层对应的 Token Mixer；得到的结果和原输入做残差相加。随后再次进行 RMSNorm，再经过 SwiGLU FFN，并进行第二次残差连接。也就是说，DeltaNet 与 Full Attention 的区别主要发生在“Token 之间怎样交换信息”这一部分，而 FFN 和残差结构保持一致。

模型同时包含一个视觉编码器。视觉编码器的隐藏维度为 1152，深度为 27 层，使用 16 个视觉注意力头，视觉 MLP 的中间维度为 4304。视觉部分最终会把图像或视频转换成 5120 维视觉 Token，随后与文本 Token 共同组成统一的多模态序列，并进入同一套语言主干。

二、文本输入与 Token Embedding

语言模型首先接收经过 Chat Template 组织后的字符串。Tokenizer 的作用是把字符串转换成离散 Token ID。设一个 batch 中包含 B 个序列，每个序列长度为 N ，那么 Tokenizer 的输出可以记为 $X_{\text{id}} \in \mathbb{N}^{B \times N}$ 。其中每一个位置只保存一个整数编号，这个整数本身并不具有连续的语义关系，例如编号相邻的两个 Token 并不意味着语义相近。

为了让神经网络能够处理这些离散编号，模型需要一个可学习的 Embedding 矩阵 $E \in \mathbb{R}^{248320 \times 5120}$ 。如果第 t 个 Token 的编号为 x_t ，那么它对应的隐藏向量就是 $h_t^{(0)} = E[x_t] \in \mathbb{R}^{5120}$ 。把所有位置的向量堆叠起来以后，得到文本隐藏状态 $H_{\text{text}}^{(0)} \in$



图 1 Qwen3.8-27B 的整体模块组成。

$\mathbb{R}^{B \times N \times 5120}$ 。

这里的 5120 是整个语言主干统一的隐藏维度。之后无论进入 DeltaNet、Full Attention 还是 SwiGLU FFN，层与层之间传递的主残差流都会保持最后一个维度为 5120。需要注意，Embedding 的作用只是把离散 Token ID 查表映射成连续向量，它并不负责不同 Token 之间的信息交换；真正的上下文建模发生在后面的 64 层 Hybrid Decoder 中。

三、视觉输入与 Vision Encoder

图片和视频不能直接进入语言模型，因为原始输入是像素，而语言主干需要的是 5120 维 Token。视觉部分首先要做的事情，就是把连续像素切分成 Patch，并把每个 Patch 转换为视觉向量。

设视觉输入写成 $I \in \mathbb{R}^{B \times 3 \times T \times H \times W}$ ，其中 T 表示时间维。对于单张图片，可以把时间维理解为退化情况；对于视频，时间维则表示连续帧。Qwen3.8 的视觉 Patch 使用 $2 \times 16 \times 16$ 的时空大小，因此 Patch Embedding 可以理解为一个三维卷积映射，把每个 $2 \times 16 \times 16$ 的局部像素块转换为 1152 维向量。

经过 Patch Embedding 后，视觉输入被表示为一串视觉 Token，可记为 $X_v^{(0)} \in \mathbb{R}^{B \times N_v \times 1152}$ 。其中 N_v 由图片或视频的实际尺寸决定。随后这些视觉 Token 进入 27 层 Vision Transformer。视觉 Transformer 内部采用注意力和 MLP 的组合，其主要作用是让不同视觉 Patch 之间交换信息，使单个 Patch 的表示逐渐从局部像素特征变成带有全局视觉上下文的特征。

经过 27 层以后，视觉特征仍然是 1152 维，而语言模型的隐藏维度是 5120，因此还需要 Vision Patch Merger。空间合并尺寸为 $s = 2$ ，也就是把空间上相邻的 2×2 个视觉 Patch 组合起来。四个 1152 维视觉向量合并后形成 4608 维局部表示，再通过 Merger 内部的投影映射到 5120 维。这样得到最终视觉 Token $H_{\text{vision}} \in \mathbb{R}^{B \times N'_v \times 5120}$ 。

这一步非常关键，因为只有当视觉 Token 与文本 Token 具有相同隐藏维度以后，二者才能进入同一个语言模型。最终模型构造统一隐藏序列 $H^{(0)} = [H_{\text{text}}^{(0)}; H_{\text{vision}}]$ 。从这里开始，语言主干把两种来源的 Token 作为同一序列中的元素进行统一处理。

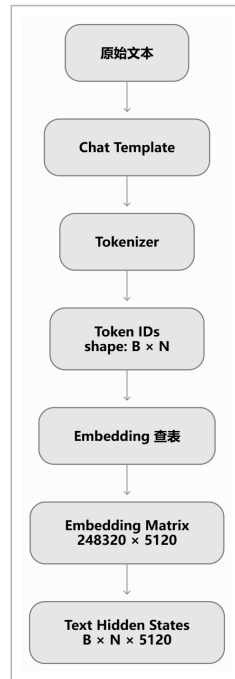


图 2 文本从 Token ID 到 5120 维隐藏表示的过程。

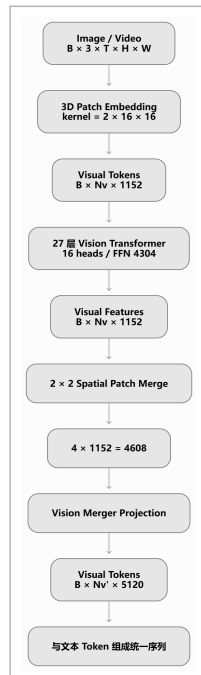


图 3 视觉输入经过 Patch Embedding、Vision Encoder 与 Patch Merger 后进入语言空间。

四、 Hybrid Decoder 的统一结构

设第 ℓ 层输入为 $H^{(\ell)} \in \mathbb{R}^{B \times N \times 5120}$ 。这一层首先做 RMSNorm，得到 $\bar{H}^{(\ell)} = \text{RMSNorm}(H^{(\ell)})$ 。这样做的原因是，随着层数增加，Residual Stream 的数值尺度可能不断变化，如果直接把未归一化的表示送入注意力或递归模块，会使训练更加不稳定。

RMSNorm 之后进入 Token Mixer。当前层如果属于四层周期中的前三层，就使用 Gated DeltaNet；如果属于第四层，就使用 Gated Full Attention。记当前 Token Mixer 为 F_ℓ ，那么它的输出为 $M^{(\ell)} = F_\ell(\bar{H}^{(\ell)})$ 。这部分负责不同 Token 之间的信息传播。

随后进行第一次残差连接，即 $U^{(\ell)} = H^{(\ell)} + M^{(\ell)}$ 。残差连接让每个模块主要学习对现有表示的增量修改，从而保留原始信息通路，并减轻深层网络中重新构造全部隐藏表示的压力。

第一次残差之后，再对 $U^{(\ell)}$ 做一次 RMSNorm，并送入 SwiGLU FFN。记 FFN 输出为 $Y^{(\ell)} = \text{FFN}(\text{RMSNorm}(U^{(\ell)}))$ ，最后得到 $H^{(\ell+1)} = U^{(\ell)} + Y^{(\ell)}$ 。因此，一个 Decoder Layer 可以理解为两个连续步骤：先做 Token Mixing，再做 Channel Mixing；前者负责位置之间的信息交换，后者负责每个 Token 内部特征的非线性变换。

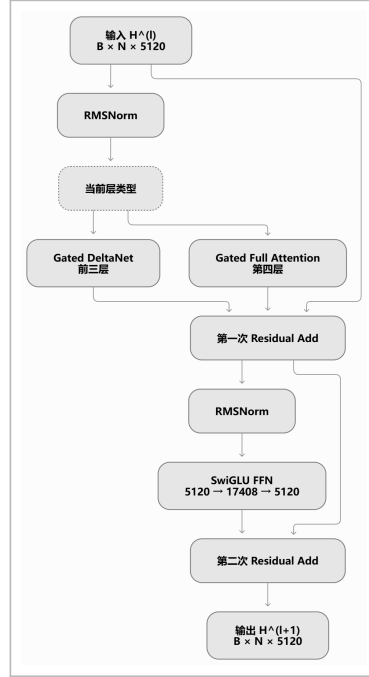


图 4 Hybrid Decoder 单层的统一结构。

五、 RMSNorm

RMSNorm 的来源可以从一个基本问题理解：神经网络中的隐藏向量如果幅值不断增大或缩小，会使后续线性层和注意力的数值范围变得难以控制。因此归一化层首先希望把向量的整体尺度调整到相对稳定的范围。

对于一个隐藏向量 $x \in \mathbb{R}^d$ ，其中 $d = 5120$ ，先计算它的均方根

$$r(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}.$$

这里的 $\epsilon = 10^{-6}$ 用来避免数值上除以零。RMSNorm 直接利用均方根控制向量尺度，不包含 LayerNorm 中的均值中心化步骤。

Qwen3.8 所属架构采用以 1 为中心的缩放参数，因此可以写成

$$\text{RMSNorm}(x) = (1 + w) \odot \frac{x}{r(x)}.$$

其中 $w \in \mathbb{R}^d$ 是可学习参数，初始化时 $w = 0$ ，所以最开始有效缩放系数就是 1。这样既保留了归一化后的基础尺度，又允许模型在训练过程中逐维调整不同特征的重要性。

六、 Gated DeltaNet

Gated DeltaNet 是 Qwen3.8-27B 最值得单独理解的模块，因为 64 个语言层中有 48 层都使用它。传统 Self-Attention 的基本做法是保留历史 Token 的 Key 和 Value，再让当前 Query 与大量历史 Key 做显式相关性计算。序列越长，需要访问和保存的历史信息就越多。

DeltaNet 维护一个固定尺寸的状态矩阵 S_t ，让这个状态逐步吸收历史信息。当前 Token 到来以后，模型先根据旧状态判断“已有记忆能够解释多少当前信息”，再把仍未被解释的误差部分写回状态。这就是 Delta Rule 的核心来源。

6.1 Q、K、V 与控制分支

设输入为 $X \in \mathbb{R}^{B \times N \times 5120}$ 。DeltaNet 首先通过线性映射得到 Query、Key 和 Value，同时还会生成用于控制状态更新和输出门控的分支。Q/K 使用 16 个 Head，每个 Head 为 128 维，因此 Q 和 K 的总宽度都是 $16 \times 128 = 2048$ ；Value 使用 48 个 Head，每个 Head 也是 128 维，因此 Value 总宽度为 $48 \times 128 = 6144$ 。

这里 Q/K Head 数和 Value Head 数并不相同。由于 $48/16 = 3$ ，每一组 Q/K 表示可以服务三个 Value Head。这样做可以在保持 Value 表达容量的同时减少 Q/K 部分的参数和计算量。

除了 Q、K、V 之外，模型还会生成 Z 、 a 和 b 。其中 Z 主要用于控制最终输出， a 与 b 则用于产生状态衰减和写入强度。也就是说，DeltaNet 不只是计算内容向量，还会动态决定“旧记忆保留多少”和“当前信息写入多少”。

6.2 短程因果卷积

Q、K、V 在进入递归状态更新之前，先经过 kernel size 为 4 的 Depthwise Causal Conv1D。这一步用较低的计算代价补充短距离局部建模。

对于某一个通道，在位置 t 的卷积结果可以理解为

$$\tilde{x}_t = \text{SiLU}(c_0 x_t + c_1 x_{t-1} + c_2 x_{t-2} + c_3 x_{t-3}).$$

因为是 Causal Conv，当前位置只能使用自己和过去三个位置，不会看到未来 Token。这样，局部相邻 Token 的模式先被卷积整合，再交给后面的递归状态负责更长距离的信息记忆。

这一结构同时包含局部卷积和递归记忆。局部卷积负责邻近 Token 的短程模式，递归状态负责更长距离的信息累积。

6.3 状态衰减与写入门

假设上一时刻的记忆状态为 S_{t-1} 。如果每一个新 Token 都只不断向状态中累加信息，那么很早以前的内容会长期残留，状态也容易逐渐饱和。因此模型首先需要遗忘机制。

模型通过 a_t 与可学习参数产生一个非正数门控 g_t ，再令 $\alpha_t = \exp(g_t)$ 。因为 $g_t \leq 0$ ，所以 α_t 位于 $(0, 1]$ 。这样旧状态先变成 $\bar{S}_{t-1} = \alpha_t S_{t-1}$ 。当 α_t 接近 1 时，旧信息基本保留；当 α_t 较小时，旧信息被更明显地衰减。

另一个门控来自 b_t ，写成 $\beta_t = \sigma(b_t)$ 。Sigmoid 把它限制在 $(0, 1)$ ，因此 β_t 可以自然解释为当前信息的写入强度。于是 α_t 主要负责“忘多少”， β_t 主要负责“写多少”。

6.4 Delta Rule 的来源

现在考虑一个 Value Head。其状态矩阵记为 $S_t \in \mathbb{R}^{128 \times 128}$ 。状态更新首先利用当前 Key k_t 查询旧状态，得到旧记忆对当前 Value 的预测结果。

旧状态给出的预测为

$$\hat{v}_t = \bar{S}_{t-1}^\top k_t.$$

如果 $\hat{v}_t$ 已经很接近真实的 v_t ，说明旧状态已经包含了这部分信息，就没有必要再次完整写入；如果二者差距很大，才需要明显修改记忆。因此定义预测误差

$$e_t = v_t - \hat{v}_t.$$

再用写入门控制修正强度，得到 $\delta_t = \beta_t e_t$ 。最后以外积形式更新状态：

$$S_t = \bar{S}_{t-1} + k_t \delta_t^\top.$$

这个外积为什么合理，可以从矩阵作用理解。 k_t 决定“在记忆空间的哪个方向进行修改”，而 δ_t 决定“这个方向上应该补充怎样的 Value 信息”。因此这一更新实际上是在修正 Key 与 Value 之间的关联关系。

把前面的衰减和误差项代入以后，也可以写成

$$S_t = \alpha_t S_{t-1} + k_t [\beta_t (v_t - (\alpha_t S_{t-1})^\top k_t)]^\top.$$

这个式子比较长，所以理解时不应该直接从合并式开始，而应当记住它实际上只有四步：旧状态先衰减，Key 从旧状态读出预测 Value，计算预测误差，然后把误差写回状态。

6.5 使用 Query 读取状态

状态更新完成以后，Query 的作用才真正出现。当前输出为

$$o_t = S_t^\top q_t.$$

因此在 DeltaNet 中，Key 主要参与“写入和定位”，Value 表示“需要写入的内容”，Query 则负责“从当前状态中读取与当前 Token 相关的信息”。

如果把全部 48 个 Value Head 放在一起，那么递归状态的形状可以写成 $S_t \in \mathbb{R}^{B \times 48 \times 128 \times 128}$ 。这个状态的大小与已经处理过多少 Token 无关，因此从缓存角度看，它和传统不断增长的 KV Cache 有本质区别。

6.6 输出门控

48 个 Value Head 的输出拼接后总宽度为 $48 \times 128 = 6144$ 。模型还有一条由输入产生的 Z 分支，用于控制这些状态读取结果最终有多少能够进入 Residual Stream。

可以把这一过程写成

$$\tilde{O} = \text{RMSNorm}(O) \odot \text{SiLU}(Z).$$

随后再通过一个线性映射把 6144 维投影回 5120 维。这里的 Gate 使模型能够根据当前 Token 的内容动态抑制或放大 DeltaNet 输出，因此这一模块称为 Gated DeltaNet。

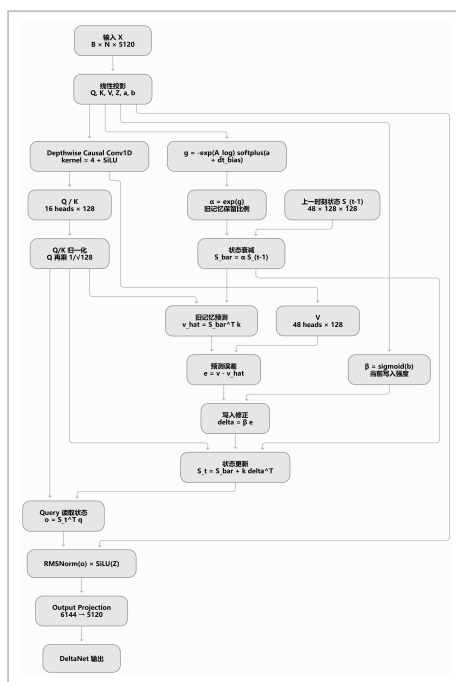


图 5 Gated DeltaNet 的局部卷积、状态衰减、误差写入和状态读取。

七、 Gated Full Attention

每四层中的第四层使用 Gated Full Attention。它的作用是周期性恢复显式的 Token-to-Token 全局联系。DeltaNet 把历史压缩到固定状态中，这种表示效率很高，但压缩意味着模型不再保留每一个历史位置的独立表示；Full Attention 则允许当前 Query 直接和所有可见历史 Key 计算相关性，因此两种模块是互补关系。

7.1 Query、Key、Value 与 GQA

Full Attention 使用 24 个 Query Head、4 个 KV Head，每个 Head 的维度为 256。于是 Query 的总内容宽度为 $24 \times 256 = 6144$ ，而 Key 和 Value 的总宽度分别为 $4 \times 256 = 1024$ 。

这里采用的是 Grouped Query Attention。因为 Query Head 有 24 个，而 KV Head 只有 4 个，所以每一个 KV Head 被 6 个 Query Head 共享。这样做最直接的好处是减少 KV Cache：模型不需要为 24 个 Query Head 分别保存 24 份 K 和 V，而只需要保存 4 份。

Q Projection 还会同时生成一组与 Query 相同形状的 Gate。也就是说，这一投影不只负责产生用于注意力计算的 Q，还会产生后面控制 Attention 输出强度的门控信号。

7.2 OK Norm

在计算 QK^T 之前, Q 和 K 会分别进行 RMSNorm。归一化沿每一个 256 维 Head 独立进行, 因此它直接控制各个注意力头内部的 Query 和 Key 尺度。

这样做的原因是注意力分数直接来自 Q 和 K 的内积。如果 Q 或 K 的幅值变化过大, QK^T 的数值范围也会明显变化, Softmax 就容易变得过于尖锐或不稳定。因此 OK Norm 的主要作用是先控制每个 Head 内部的尺度, 再进行相关性计算。

7.3 Partial MRoPE

每个 Attention Head 有 256 维，其中 64 维参与旋转位置编码，其余 192 维保持普通内容表示。于是一个 Query Head 可以拆成 $q_{\text{rot}} \in \mathbb{R}^{64}$ 和 $q_{\text{pass}} \in \mathbb{R}^{192}$ 两部分。

位置编码只作用在前 64 维上，即 $q'_{\text{rot}} = R(p)q_{\text{rot}}$ ，其余 192 维保持不变，再把两部分重新拼接。Key 采用同样处理。这样模型既保留一部分显式位置敏感维度，也保留较大的非旋转内容空间。

对于纯文本，位置主要是一维顺序；对于图像和视频，则需要同时表达时间、横向和纵向位置，因此这里使用多模态 RoPE，使视觉 Token 的空间和时间关系能够进入同一套 Attention 位置机制。

7.4 缩放点积注意力

经过 QK Norm 和位置编码以后，对第 h 个 Query Head 计算

$$S_h = \frac{Q_h K_h^\top}{\sqrt{256}} = \frac{Q_h K_h^\top}{16}.$$

分母 $\sqrt{256}$ 来自标准 Scaled Dot-Product Attention。原因是当 Head 维度较大时，未经缩放的点积方差会随维度增大，使 Softmax 更容易进入饱和区域，因此需要除以 $\sqrt{d_h}$ 来稳定分数尺度。

因为语言模型是自回归生成模型，还必须加入 Causal Mask。对于位置 i ，只能访问 $j \leq i$ 的历史位置，而不能看到未来 Token。加入 Mask 后再计算 Softmax 得到注意力概率 A_h ，最后使用这些概率对 Value 加权求和，得到 $O_h = A_h V_h$ 。

24 个 Query Head 的输出拼接后得到 6144 维 Attention 表示。

7.5 Attention Output Gate

Full Attention 的输出先使用 Q Projection 同时产生的 Gate 进行调制，再投影回 5120 维。可以写成

$$\tilde{O} = O \odot \sigma(G).$$

Sigmoid 把每一个门控值限制在 0 到 1 之间，因此模型可以根据当前输入决定哪些 Attention 通道应该被保留，哪些应该被抑制。

最后通过输出矩阵把 6144 维投影回 5120 维，再和 Residual Stream 相加。这样，Full Attention 完成的是一次显式全局交互，而 Gate 控制这次全局信息应该以多大强度写回当前隐藏状态。

八、 SwiGLU 前馈网络

无论当前层使用 DeltaNet 还是 Full Attention，Token Mixer 之后的 FFN 都使用 SwiGLU。理解 SwiGLU 时，可以先从普通两层 MLP 出发：普通 FFN 通常先把 5120 维扩展到更大的中间维度，再通过非线性激活，最后投影回 5120 维。

SwiGLU 使用两个并行上投影分支。设输入为 $X \in \mathbb{R}^{B \times N \times 5120}$ ，两个投影分别得到 $G = XW_g$ 和 $U = XW_u$ ，二者的最后一维都为 17408。随后门控分支经过 SiLU，得到 $\text{SiLU}(G)$ ，再与另一条分支逐元素相乘，最后通过 Down Projection 返回 5120 维：

$$\text{FFN}(X) = [\text{SiLU}(XW_g) \odot (XW_u)] W_d.$$

这里的来源可以理解成“内容分支”和“门控分支”的结合。 U 提供候选特征，而 $\text{SiLU}(G)$ 决定每个中间特征应该通过多少。相比单纯的 ReLU 或 GELU MLP，SwiGLU 让 FFN 自己学习更加细粒度的特征选择。

因此 Token Mixer 和 FFN 的角色不同。DeltaNet 或 Attention 主要改变一个 Token 能够从其他位置获取什么信息，而 SwiGLU 主要改变已经得到的信息在当前 Token 内部怎样重新组合。

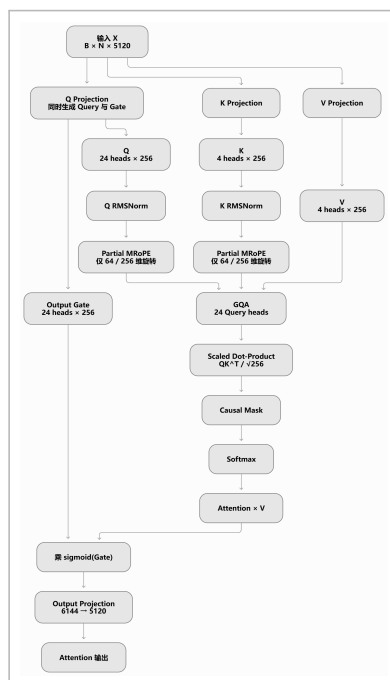


图 6 Gated Full Attention 中的 QK Norm、Partial MRoPE、GQA 和输出门控。

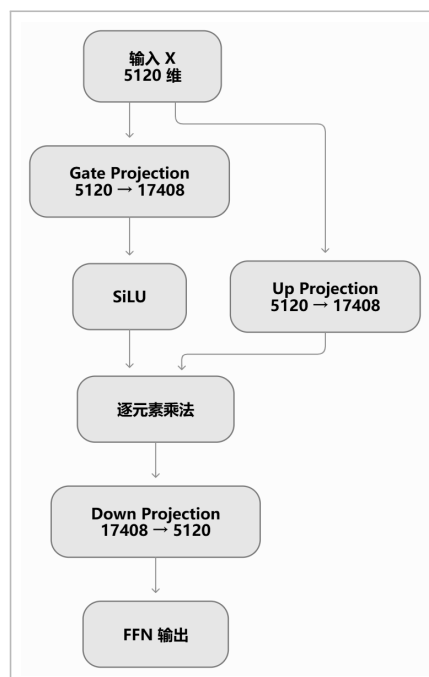


图 7 SwiGLU 前馈网络的双分支门控结构。

九、为什么采用三层 DeltaNet 加一层 Full Attention

如果 64 层全部使用 Full Attention，那么序列长度为 N 时，每一层都需要构造与 N^2 相关的 Token 两两关系，并且推理时需要持续保存随上下文长度增长的 KV Cache。对于 262144 Token 甚至更长的上下文，这种代价会非常明显。

DeltaNet 把长期历史压缩到固定尺寸状态矩阵中。新 Token 到来以后，只需要更新状态并读取状态，因此每个 DeltaNet 层无需重新访问完整历史序列，这使它特别适合长上下文。

固定尺寸状态会对历史信息进行压缩。当当前 Token 需要精确访问很早以前的某个具体位置时，Full Attention 的显式位置访问更加直接。因此 Qwen3.8 每隔三层 DeltaNet 插入一层 Full Attention，用来周期性恢复全局 Token 之间的直接联系。

这种排列可以理解为：连续三层先进行高效的局部建模和递归记忆更新，第四层再进行一次全局校正和显式长距离交互，然后进入下一个四层周期。整体设计在高效压缩记忆与精确全局访问之间保持平衡。

十、推理缓存

推理时，Qwen3.8-27B 的两种 Token Mixer 需要保存的历史信息不同。

Full Attention 层需要保存历史 Key 和 Value。由于它使用 4 个 KV Head、每个 Head 维度为 256，因此每一个历史 Token 在单层中需要保存的 K/V 元素数为 $2 \times 4 \times 256 = 2048$ 。随着生成长度增加，这部分 Cache 会近似线性增长。

DeltaNet 层不保存完整历史 K/V，而主要保存固定尺寸递归状态 S_t 和很短的 Causal Conv State。它的状态大小不随已经处理的 Token 数量持续增加。因此，从整个 64 层主干看，只有 16 个 Full Attention 层维护传统长度相关的 KV Cache，而另外 48 层主要维护固定大小状态。

这也是混合架构在长上下文场景中的一个重要工程意义：它同时减少了 Full Attention 的计算次数，并降低了大量层随序列长度增长的长期缓存压力。

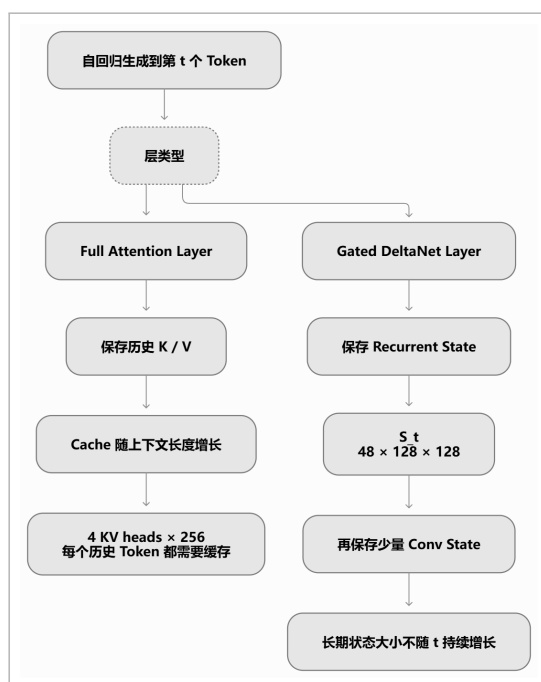


图 8 Full Attention KV Cache 与 DeltaNet 固定递归状态的差异。

十一、 Final RMSNorm 与 LM Head

经过 64 层语言主干以后，得到 $H^{(64)} \in \mathbb{R}^{B \times N \times 5120}$ 。模型首先再做一次 Final RMSNorm，使送入输出层的隐藏表示保持稳定尺度。

如果当前要预测序列中下一个 Token，只需要取当前位置对应的隐藏向量 $h_t \in \mathbb{R}^{5120}$ 。LM Head 是一个从 5120 维映射到整个词表的线性层，其输出为 $z_t \in \mathbb{R}^{248320}$ 。这里每一个分量都对应一个词表 Token 的 Logit。

Logit 表示未归一化的词表分数。需要得到概率分布时，可以计算

$$P(x_{t+1} = i \mid x_{\leq t}) = \frac{\exp(z_{t,i})}{\sum_{j=1}^{248320} \exp(z_{t,j})}.$$

推理系统再根据 Greedy、Temperature、Top- k 或 Top- p 等策略选择下一个 Token。新 Token 被加入上下文以后，模型继续预测下一个位置，于是整个生成过程不断重复。

从概率模型角度看，自回归语言模型最终学习的是联合概率的分解

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

前面所有的 Vision Encoder、DeltaNet、Full Attention、SwiGLU 和 RMSNorm，最终都服务于同一个目标：为当前位置产生更好的隐藏状态，从而提高下一个 Token 条件概率的质量。

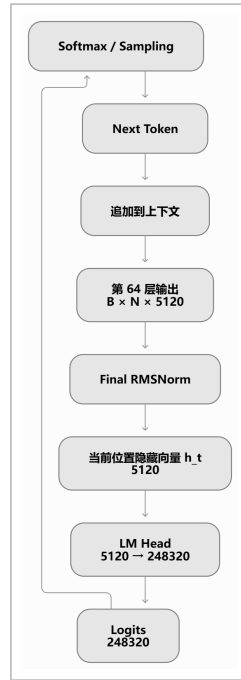


图 9 Final RMSNorm、LM Head 与自回归生成。

十二、 MTP 与主干结构

Qwen3.8-27B 还包含 Multi-Token Prediction 相关训练结构。普通自回归训练主要要求当前位置隐藏状态预测下一个 Token，而 MTP 会进一步加入更远未来 Token 的预测监督。

它的意义主要在训练侧。只预测一步时，隐藏状态主要围绕 x_{t+1} 建立预测能力；加入多步未来目标以后，当前表示还必须包含更多能够支持后续 Token 的信息，因此训练信号会更加丰富。

需要把 MTP 和主干架构区分开。MTP 并不意味着模型内部又存在一套新的 27B 网络，语言主干仍然是前面介绍的 64 层 Hybrid Decoder。Thinking、Non-Thinking 等行为同样不代表存在两套独立模型，它们主要属于后训练、输入模板和生成控制层面的差异。

十三、 模型结构总结

Qwen3.8-27B 的整个计算过程可以沿着真实数据流理解。文本首先经过 Tokenizer 和 Token Embedding 变成 5120 维语言 Token；图片和视频经过三维 Patch Embedding、27 层 Vision Encoder 和 Patch Merger，最终同样变成 5120 维视觉 Token。两种模态组成统一序列以后，进入 64 层 Hybrid Decoder。

在语言主干内部，每四层形成一个周期。前三层使用 Gated DeltaNet：短程因果卷积先处理局部 Token 模式，然后通过状态衰减、预测误差和 Delta Rule 更新固定尺寸记忆，再由 Query 读取当前状态。第四层使用 Gated Full Attention：Q/K 先做归一化，再加入 Partial MRoPE，通过 24 个 Query Head 和 4 个 KV Head 完成 GQA，之后进行因果缩放点积注意力，并利用输出 Gate 调节写回 Residual Stream 的强度。

每一个 Token Mixer 后面都接相同的 SwiGLU FFN。经过 64 层以后，模型再通过 Final RMSNorm 和 LM Head 产生整个词表上的 Logit，并以自回归方式不断生成后续 Token。

因此，Qwen3.8-27B 的关键在于把视觉编码、局部卷积、递归状态记忆、周期性全局 Attention 和门控 FFN 组合在同一个统一架构中。其中 DeltaNet 主要解决长序列效率问题，Full Attention 保留显式全局交互能力，Vision Encoder 则负责把图像和视频转换到与语言统一的 Token 表示空间。